

Body Domain Controller (BDC)

High-Level Design Document

Field	Value
Document ID	BDC-HLD-001
Document Title	Body Domain Controller High-Level Design
Version	1.0
Date	2026-09-15
Status	Approved (synthetic dataset)

1 Introduction

1.1 Purpose and Scope

This document describes the High-Level Design (HLD) of the Body Domain Controller (BDC) application software. The BDC is a fictitious ECU created as a synthetic dataset for academic evaluation of document analysis tools. The software is based on AUTOSAR Classic Platform release R22-11. It implements central locking, power windows and exterior lighting.

1.2 Reference Documents

The AUTOSAR Classic Platform software component template and RTE specification are used for terminology only. No standards text is reproduced in this synthetic document.

1.3 Glossary

Table 1-1: Glossary

Term	Meaning
SWC	Software Component
RTE	Runtime Environment
BSW	Basic Software
NVM	Non-Volatile Memory
DID	Data Identifier
NRC	Negative Response Code
P-Port	Provided port of a software component
R-Port	Required port of a software component

2 System Overview

2.1 System Context

The BDC receives remote commands from the key fob receiver and the vehicle speed from the ESC ECU. It controls the door locks, the power windows and the exterior lamps.

2.2 Platform

The BDC communicates on a single high-speed CAN network with a bit rate of 500 kbit/s. The application software runs on a synthetic 32-bit microcontroller under an AUTOSAR Classic Platform operating system. Application components communicate only through the Runtime Environment (RTE).

3 Software Architecture

3.1 Architecture Overview

The application layer of the BDC consists of seven software components (SWC) that communicate through the RTE. Figure 3-1 shows the components and their relationships. The formal port and connection definitions are given in Sections 4 and 6.

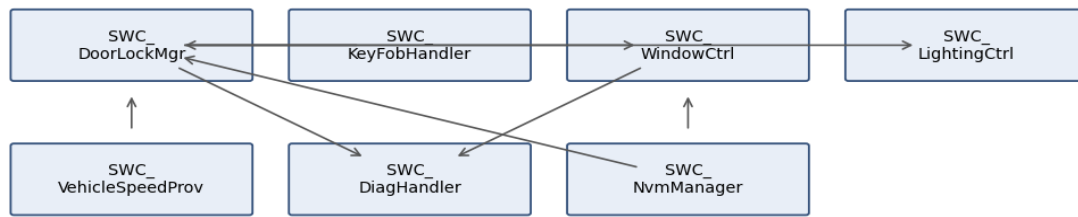


Figure 3-1: BDC software component overview

3.2 Component Catalogue

Table 3-1: Component catalogue

Component	Type	Responsibility
SWC_DoorLockMgr	Application	Central locking state machine and lock arbitration
SWC_KeyFobHandler	Application	Remote key fob command decoding and validation
SWC_WindowCtrl	Application	Power window control with anti-pinch protection
SWC_LightingCtrl	Application	Exterior lighting and courtesy lamp control
SWC_VehicleSpeedProv	Application	Filtered vehicle speed provider
SWC_DiagHandler	Service	UDS diagnostic request routing
SWC_NvmManager	Service	Non-volatile memory access service

3.3 Component Descriptions

3.3.1 SWC_DoorLockMgr

SWC_DoorLockMgr implements the central locking state machine with the states LOCKED, UNLOCKED and SUPERLOCKED. It arbitrates lock and unlock requests from SWC_KeyFobHandler and from the auto-lock function. Auto-lock is triggered when VehicleSpeed exceeds 15 km/h while any door is unlocked. The component persists the last lock state through SWC_NvmManager every time the state changes. Runnable DoorLockMgr_Run10ms executes every 10 ms.

3.3.2 SWC_KeyFobHandler

SWC_KeyFobHandler decodes remote key fob commands received on the CAN message KeyFob_Cmd. It validates the rolling-code counter supplied by the immobilizer and discards commands with an invalid counter. Valid commands are forwarded to SWC_DoorLockMgr as KeyFobCmd. Runnable KeyFobHandler_Run20ms executes every 20 ms.

3.3.3 SWC_WindowCtrl

SWC_WindowCtrl controls the four power windows including one-touch up and down movement. Anti-pinch protection stops and reverses the window within 50 ms of pinch detection. The comfort-close function raises all windows when the vehicle is locked with the key fob held for more than 2 seconds, using DoorLockState from SWC_DoorLockMgr. The window position is transmitted as WindowPosition. Runnable WindowCtrl_Run10ms executes every 10 ms.

3.3.4 SWC_LightingCtrl

SWC_LightingCtrl controls the exterior lighting: low beam, high beam, turn indicators and the courtesy lamp. The courtesy lamp is switched on for 30 seconds when SWC_DoorLockMgr reports an unlocked state. Turn indicators flash at 1.5 Hz. The component requires VehicleSpeed to enable the speed-dependent cornering lamp. Runnable LightingCtrl_Run50ms executes every 50 ms.

3.3.5 SWC_VehicleSpeedProv

SWC_VehicleSpeedProv provides the filtered vehicle speed derived from the wheel-speed message VehSpd_ESC received from the ESC ECU. The value is a 16-bit unsigned integer with a resolution of 0.01 km/h. Signals with a failed CRC are replaced by the last valid value for up to 100 ms, after which the output is flagged invalid. Runnable VehSpeedProv_Run20ms executes every 20 ms.

3.3.6 SWC_DiagHandler

SWC_DiagHandler routes UDS diagnostic requests received from the diagnostic communication manager to the owning component. It supports the services ReadDataByIdentifier (0x22) and WriteDataByIdentifier (0x2E) for the data identifiers defined in Section 4.3. Requests for unsupported identifiers are answered with negative response code (NRC) 0x31. The component is event-triggered and has no cyclic runnable.

3.3.7 SWC_NvmManager

SWC_NvmManager provides non-volatile storage services through the client-server interface If_NvmAccess. It manages three NVM blocks: LockStateBlock (4 bytes), WindowCalibBlock (16 bytes) and DtcBlock (64 bytes). Writes are queued and executed asynchronously by the memory stack; a write completes within 20 ms under normal conditions.

4 Port and Interface Specification

4.1 Interface Definitions

Table 4-1: Interface definitions

Interface	Kind	Data Element / Operation	Data Type
If_KeyFobCmd	Sender-Receiver	KeyFobCmd	uint8
If_VehicleSpeed	Sender-Receiver	VehicleSpeed	uint16
If_DoorState	Sender-Receiver	DoorLockState	uint8
If_WindowPos	Sender-Receiver	WindowPosition	uint8
If_LampCmd	Sender-Receiver	LampCmd	uint8
If_NvmAccess	Client-Server	ReadBlock	Std_ReturnType
If_NvmAccess	Client-Server	WriteBlock	Std_ReturnType
If_DiagData	Client-Server	ReadDid	Std_ReturnType
If_DiagData	Client-Server	WriteDid	Std_ReturnType

4.2 Port Specification

Table 4-2: Port specification

Component	Port	Direction	Interface
SWC_KeyFobHandler	PP_KeyFobCmd	P-Port	If_KeyFobCmd
SWC_DoorLockMgr	RP_KeyFobCmd	R-Port	If_KeyFobCmd
SWC_DoorLockMgr	RP_VehicleSpeed	R-Port	If_VehicleSpeed
SWC_DoorLockMgr	PP_DoorState	P-Port	If_DoorState
SWC_DoorLockMgr	RP_NvmAccess	R-Port	If_NvmAccess
SWC_DoorLockMgr	PP_DiagData	P-Port	If_DiagData
SWC_WindowCtrl	RP_DoorState	R-Port	If_DoorState
SWC_WindowCtrl	PP_WindowPos	P-Port	If_WindowPos
SWC_WindowCtrl	RP_NvmAccess	R-Port	If_NvmAccess
SWC_WindowCtrl	PP_DiagData	P-Port	If_DiagData
SWC_LightingCtrl	RP_DoorState	R-Port	If_DoorState
SWC_LightingCtrl	RP_VehicleSpeed	R-Port	If_VehicleSpeed

Component	Port	Direction	Interface
SWC_LightingCtrl	PP_LampCmd	P-Port	If_LampCmd
SWC_VehicleSpeedProv	PP_VehicleSpeed	P-Port	If_VehicleSpeed
SWC_DiagHandler	RP_DiagDoor	R-Port	If_DiagData
SWC_DiagHandler	RP_DiagWindow	R-Port	If_DiagData
SWC_NvmManager	PP_NvmAccess	P-Port	If_NvmAccess

4.3 Diagnostic Data Identifiers

Table 4-3: Diagnostic data identifiers

DID	Name	Owner Component	Length (bytes)
0x0100	DoorLockCounter	SWC_DoorLockMgr	2
0x0101	LastLockState	SWC_DoorLockMgr	1
0x0200	WindowCalibStatus	SWC_WindowCtrl	1

5 Signal Specification

5.1 CAN Signals

All signals are transmitted in Motorola byte order. The cycle time Event denotes event-triggered transmission.

Table 5-1: CAN signal specification

Signal	CAN Message	CAN ID	Length (bit)	Dir	Cycle (ms)	Mapped Port
KeyFobCmd	KeyFob_Cmd	0x2A0	8	Rx	Event	SWC_KeyFobHandler.PP_KeyFobCmd
VehicleSpeed	VehSpd_ESC	0x1F0	8	Rx	20	SWC_VehicleSpeedProv.PP_VehicleSpeed
DoorLockState	BDC_Status	0x3B0	8	Tx	100	SWC_DoorLockMgr.PP_DoorState
WindowPosition	BDC_Window	0x3B1	8	Tx	100	SWC_WindowCtrl.PP_WindowPos
LampCmd	BDC_Lamp	0x3B2	8	Tx	50	SWC_LightingCtrl.PP_LampCmd

6 Dependencies and Connections

6.1 Connection Table

Every required port (R-Port) must be connected to exactly one provided port (P-Port) with a matching interface.

Table 6-1: Connection table

Provider Port	Requirer Port	Purpose
SWC_KeyFobHandler.PP_KeyFobCmd	SWC_DoorLockMgr.RP_KeyFobCmd	Forward validated remote commands
SWC_VehicleSpeedProv.PP_VehicleSpeed	SWC_DoorLockMgr.RP_VehicleSpeed	Auto-lock threshold evaluation
SWC_DoorLockMgr.PP_DoorState	SWC_WindowCtrl.RP_DoorState	Comfort close
SWC_DoorLockMgr.PP_DoorState	SWC_LightingCtrl.RP_DoorState	Courtesy lamp control
SWC_NvmManager.PP_NvmAccess	SWC_DoorLockMgr.RP_NvmAccess	Persist lock state
SWC_NvmManager.PP_NvmAccess	SWC_WindowCtrl.RP_NvmAccess	Persist window calibration
SWC_DoorLockMgr.PP_DiagData	SWC_DiagHandler.RP_DiagDoor	Diagnostic data access to door lock data
SWC_WindowCtrl.PP_DiagData	SWC_DiagHandler.RP_DiagWindow	Diagnostic data access to window data

7 Functional Flows

7.1 FF-01 Remote Unlock

- Step 1: The Com stack receives the CAN message KeyFob_Cmd and updates the signal KeyFobCmd.
- Step 2: SWC_KeyFobHandler validates the rolling-code counter and discards invalid commands.
- Step 3: SWC_KeyFobHandler publishes KeyFobCmd with value 2 (unlock) through port PP_KeyFobCmd.
- Step 4: SWC_DoorLockMgr transitions from the state LOCKED to the state UNLOCKED.
- Step 5: SWC_DoorLockMgr writes the new state through SWC_NvmManager.
- Step 6: SWC_DoorLockMgr publishes DoorLockStatus to SWC_WindowCtrl and SWC_LightingCtrl.
- Step 7: The Com stack transmits BDC_Status within 100 ms of Step 1.

7.2 FF-02 Speed-Dependent Auto-Lock

- Step 1: SWC_VehicleSpeedProv publishes VehicleSpeed every 20 ms.
- Step 2: SWC_DoorLockMgr compares VehicleSpeed with the auto-lock threshold of 15 km/h.
- Step 3: If the threshold is exceeded and any door is unlocked, SWC_DoorLockMgr transitions to the state LOCKED.
- Step 4: SWC_ClimateCtrl is notified to close the air vents.
- Step 5: SWC_DoorLockMgr publishes the new DoorLockState.

7.3 FF-03 Window Anti-Pinch Reversal

- Step 1: SWC_WindowCtrl detects a pinch event from the motor current while a window is closing.
- Step 2: SWC_WindowCtrl stops the motor and reverses the window within 50 ms.
- Step 3: SWC_WindowCtrl publishes the new WindowPosition.

7.4 FF-04 Diagnostic Data Read

- Step 1: A UDS request with service 0x22 for a data identifier is received by SWC_DiagHandler.
- Step 2: SWC_DiagHandler routes the request to the owning component through the port RP_DiagDoor or RP_DiagWindow.
- Step 3: The owning component returns the data and SWC_DiagHandler sends a positive response with service 0x62.
- Step 4: For an unsupported identifier SWC_DiagHandler sends negative response code 0x31.

8 Non-Functional Requirements

8.1 Timing and Performance

Table 8-1: Non-functional requirements

ID	Requirement	Value
NFR-01	Central lock actuation latency from KeyFobCmd reception	max 100 ms
NFR-02	Anti-pinch reversal time after pinch detection	max 50 ms
NFR-03	Auto-lock vehicle speed threshold	15 km/h
NFR-04	NVM write completion time	max 20 ms
NFR-05	Courtesy lamp on-time after unlock	30 s
NFR-06	Vehicle speed signal hold time after CRC failure	max 100 ms

9 Revision History

Table 9-1: Revision history

Version	Date	Change
1.0	2026-09-15	Initial release of the BDC high-level design.